作文自动评分系统集成测试计划文档

1. 文档目标

本文档旨在规范和指导作文自动评分系统的集成测试阶段工作。主要目标包括：

- 验证 Flask API、模型服务 (`EssayScoringService`) 与 SQLite 存储 (`HistoryStore`) 之间的链路连通性。
- 确保前后端分离架构下，Streamlit 前端发起的 HTTP 请求能够被后端正确处理并返回符合约定的 JSON 数据。
- 暴露系统在组件组装后可能产生的数据丢失、状态异常、接口超时或并发处理等问题。

2. 测试范围与策略

2.1 测试范围

- **后端内部集成**：API 路由控制层 -> `EssayScoringService`（模型推理） -> `HistoryStore`（数据持久化）。
- **前后端 API 集成**：Streamlit 页面逻辑 -> 内部封装的 API 调用函数 -> Flask RESTful 接口。

2.2 测试策略

- **测试框架**：使用 `pytest` 结合 Flask 的 `test_client` 进行后端接口集成测试。
- **数据库策略**：使用独立于生产环境的测试数据库文件（如 `test_history.db`），并在每次测试会话前后执行数据清理（Setup / Teardown），确保测试用例之间的独立性。
- **模型加载策略**：由于加载真实的 PyTorch 模型 (`best_fold_model.pt`) 耗时较长，集成测试环境中可配置加载一个**轻量级/随机初始化的同结构 Dummy 模型**，或者在全局 Setup 阶段仅加载一次真实模型供所有用例复用。

---

3. 测试环境配置

环境组件	配置说明
测试数据库	`data/test_history.db`（每次跑完自动化脚本后清空）
模型文件	`model/dummy_model.pt`（结构与原模型相同，用于快速跑通链路）
测试工具	`pytest`, `requests` (前端 API 测试), Flask Test Client
文件归档	独立设置 `uploads/test_uploads/` 目录，验证文件 I/O 权限

---

4. 集成测试场景设计

4.1 核心主干链路：单篇评分 (API + Model + DB)

测试目标：验证从接收评分请求、调用模型算分、提取文本特征、入库到返回完整结果的单向完整数据流。

场景 ID	测试说明	步骤说明	预期结果
`INT-SCORE-01`	**标准题目单篇评分**	1. 客户端发送 POST `/api/score/text`，携带有效文本和 `prompt_id`。 2. 系统调用模型计算。 3. 系统将原始文本、特征及分数写入测试 DB。	HTTP 200；返回包含 `score` 和 `submission_id` 的 JSON；测试 DB 的主表与记录表各新增 1 条数据，且数据字段对应一致。
`INT-SCORE-02`	`unknown` **题目综合评分**	1. 客户端请求携带 `prompt_id=unknown`。 2. 触发多题目集合的推理与均值融合策略。	返回的分数在 0-100 之间；DB 中记录的 `prompt_id` 为 `unknown`。
`INT-SCORE-03`	**极端长文本评分链路**	发送超出模型 `max_length` (如 4000 字符) 的文本请求。	模型能自动截断或系统捕获异常返回 400/500，不会导致 Flask 进程崩溃，数据库状态一致（不产生脏数据）。

4.2 核心主干链路：批量评分文件处理 (API + File I/O + DB)

测试目标：验证 CSV 文件上传归档、后端解析、循环调用模型与批量存库的复杂链路。

场景 ID	测试说明	步骤说明	预期结果
`INT-BATCH-01`	**标准 CSV 批量上传评分**	1. 模拟 `multipart/form-data` 上传包含 5 篇作文的 CSV。 2. 后端保存文件到 `uploads` 测试目录。 3. 解析执行并入库。	HTTP 200；本地 `uploads` 目录成功生成带时间戳的文件；返回含 5 条结果的 JSON；DB 主表增加 1 条记录，子表增加 5 条记录。
`INT-BATCH-02`	**非法文件格式拦截**	上传一个非 CSV/TXT 的二进制文件（如 `.png` 或伪装后缀文件）。	HTTP 400 报错提示格式错误；本地不残留无效上传文件；DB 数据行数无变化。

4.3 状态管理链路：历史记录管理 (API + DB)

测试目标：验证查询接口的分页逻辑、数据拼装逻辑，以及删除接口的级联删除有效性。

场景 ID	测试说明	步骤说明	预期结果
`INT-HIST-01`	**历史列表分页与详情流转**	1. 前置：通过接口产生 15 条历史主记录。 2. GET `/api/history?page=2&size=10` 3. 提取其中一个 `submission_id` 请求详情。	列表接口返回 5 条数据和总数 15；详情接口返回的状态码为 200，且包含文本分析等 JSON 反序列化后的结构化数据。
`INT-HIST-02`	**历史记录级联删除清理**	1. DELETE `/api/history/<submission_id>` 删除指定的批量提交记录。 2. 直接查询测试数据库的子表（records）。	接口返回 HTTP 200/204 成功状态；DB 中关联该 `submission_id` 的所有子记录被完全清除（触发了 SQLite 级联操作或代码级联逻辑）。

4.4 前端与后端集成 (Streamlit + Flask)

测试目标：确保前端调用的函数参数结构与后端接口契合，不发生序列化/反序列化失败。

(注：通常使用自动化脚本如 Selenium/Playwright 或直接测试前端内部封装的 `call_\*` 方法)

场景 ID	测试说明	步骤说明	预期结果
`INT-FE-01`	**前端状态检测探测**	执行前端的 `call_health()` 方法连接运行中的 Flask 测试实例。	返回正确的 `is_healthy=True`，能正确提取后端的 `supported_prompts` 并渲染到界面的下拉框中。
`INT-FE-02`	**前端导出 CSV 数据对齐**	1. 调用 `call_history_detail` 获取详情 JSON。 2. 执行 `build_export_csv` 函数。	生成的 CSV 字符串各列完全对齐，特殊字符和高频词 JSON 字符串没有破坏 CSV 的逗号分隔符规则。

---

5. 执行流程与持续集成 (CI) 建议

1. **环境拉起**：在执行集成测试前，启动一个专用的 Flask 进程，挂载测试配置。
2. **测试数据准备 (Fixtures)**：
    - 在全局 `conftest.py` 中编写 `init_test_db` fixture，测试开始前建表。
    - 编写 `cleanup_uploads` fixture，测试结束后清空测试归档目录。
3. **断言原则**：除了断言 HTTP 状态码和响应报文内容，**必须**进入数据库层断言数据的实际变更情况。

6. 风险与注意事项

- **依赖项耗时**：如果未替换 `dummy` 模型，频繁拉起模型将导致测试执行极慢。建议集成测试尽量重用已实例化的 `EssayScoringService`。
- **并发限制**：SQLite 默认对并发写入支持有限。如果后续增加多线程测试场景，可能会遇到 `database is`

`locked` 的错误。在集成测试中应包含模拟并发发压的测试用例以验证当前的系统瓶颈。